

70. Climbing Stairs Easy

Acceptance: 53.9% | [LeetCode](#)

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

```
**Input:** n = 2
**Output:** 2
**Explanation:** There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps
```

Example 2:

```
**Input:** n = 3
**Output:** 3
**Explanation:** There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step
```

Constraints:

- $1 \leq n \leq 45$

Claude Opus Solution

Climbing Stairs — Solution Walkthrough

1. Intuition

The **key insight** is to think about *how you arrive at step n* . There are only two possible last moves: - You took a **single step** from step $n-1$, or - You took a **double step** from step $n-2$.

This means:

```
ways(n) = ways(n-1) + ways(n-2)
```

This is exactly the **Fibonacci recurrence**! The number of ways to reach step n is the sum of the ways to reach the two preceding steps. Once you recognize this pattern, the problem reduces to computing a Fibonacci-like sequence.

Base cases: - $\text{ways}(1) = 1$ (only one way: a single step) - $\text{ways}(2) = 2$ (either two single steps, or one double step)

2. Approach (Pseudocode)

Since each value only depends on the **previous two values**, we don't need an array — just two rolling variables.

```
if n <= 2:
    return n                # ways(1)=1, ways(2)=2 conveniently equal n

prev2 = 1                   # ways(1)
prev1 = 2                   # ways(2)

for i from 3 to n:
    current = prev1 + prev2  # ways(i) = ways(i-1) + ways(i-2)
    prev2 = prev1           # shift window forward
    prev1 = current

return prev1                # holds ways(n)
```

3. Python Solution

```
class Solution:
    def climbStairs(self, n: int) -> int:
        # Base cases:
        # n=1 -> 1 way, n=2 -> 2 ways.
        # Conveniently, for n <= 2 the answer equals n itself.
        if n <= 2:
            return n

        # 'prev2' holds ways(i-2), 'prev1' holds ways(i-1).
        # Initialize them to ways(1) and ways(2).
        prev2 = 1 # ways(1)
```

```

prev1 = 2 # ways(2)

# Iteratively build up from step 3 to step n.
# Each step only needs the two previous results.
for _ in range(3, n + 1):
    # The number of ways to reach the current step is the sum
    # of the ways to reach the two steps directly before it.
    current = prev1 + prev2

    # Slide the rolling window forward by one step:
    # the old 'prev1' becomes the new 'prev2',
    # and 'current' becomes the new 'prev1'.
    prev2 = prev1
    prev1 = current

# After the loop, 'prev1' holds ways(n).
return prev1

```

4. Complexity Analysis

Metric	Value	Explanation
Time	$O(n)$	We perform a single pass from step 3 to n , doing constant work each iteration.
Space	$O(1)$	We only ever store two variables (<code>prev1</code> , <code>prev2</code>) regardless of input size.

This improves on the reference solutions, which use $O(n)$ space (either a memo dictionary or a `memo` array). The recurrence only needs the last two values, so we eliminate the storage entirely.

5. Alternative Approaches

- Top-down recursion + memoization** (the reference solution):
- Recurse `climbStairs(n-1) + climbStairs(n-2)` and cache results.
- Time $O(n)$, Space $O(n)$ (call stack + memo).
- Cleaner to write but uses more memory and risks stack depth issues.
- Bottom-up DP with an array:**
- Build a `dp[]` table where `dp[i] = dp[i-1] + dp[i-2]`.
- Time $O(n)$, Space $O(n)$. The array is unnecessary since only two cells matter — which leads to the optimized solution above.
- Closed-form (Binet's formula):**
- Since this is Fibonacci, you can use the golden-ratio formula: `ways(n) = round( $\phi^{n+1} / \sqrt{5}$ )`, where $\phi = (1 + \sqrt{5}) / 2$
- Time $O(1)$ (treating power as constant), but suffers from floating-point precision issues for large n .
- Matrix exponentiation:**
- Compute Fibonacci via `[[1,1],[1,0]]^n` using fast exponentiation.
- Time $O(\log n)$ — overkill here given $n \leq 45$, but valuable for very large n .

Recommended choice: The iterative $O(1)$ -space solution. It's simple, fast, and memory-optimal for the given constraints.

Codex Solution

1. Intuition

At any step n , the last move must have been either:

- A 1-step move from step $n - 1$
- A 2-step move from step $n - 2$

So the number of ways to reach step n is:

$$\text{ways}(n) = \text{ways}(n - 1) + \text{ways}(n - 2)$$

This is exactly the Fibonacci pattern.

Base cases:

```
ways(1) = 1
ways(2) = 2
```

Because:

- For `n = 1`: `[1]`
- For `n = 2`: `[1, 1]`, `[2]`

2. Approach

We can compute the answer iteratively using only two variables.

Instead of storing the whole DP array, we only need the previous two values:

```
prev2 = ways(i - 2)
prev1 = ways(i - 1)
current = ways(i)
```

Pseudocode

```
function climbStairs(n):
    if n == 1:
        return 1

    prev2 = 1 // ways to reach step 1
    prev1 = 2 // ways to reach step 2

    for step from 3 to n:
        current = prev1 + prev2
        prev2 = prev1
        prev1 = current

    return prev1
```

3. Python Solution

```
class Solution:
    def climbStairs(self, n: int) -> int:
        # Base case:
        # There is only one way to climb 1 step: [1]
        if n == 1:
            return 1

        # ways_to_prev2 represents ways(i - 2)
        # ways_to_prev1 represents ways(i - 1)
        #
        # For step 1: 1 way -> [1]
        # For step 2: 2 ways -> [1, 1], [2]
        ways_to_prev2 = 1
        ways_to_prev1 = 2

        # Start computing from step 3 up to step n
        for step in range(3, n + 1):
            # To reach the current step:
            # 1. Come from previous step using a 1-step move
            # 2. Come from two steps behind using a 2-step move
            current_ways = ways_to_prev1 + ways_to_prev2

            # Move the window forward:
            # The old ways_to_prev1 becomes the new ways_to_prev2
            # The current result becomes the new ways_to_prev1
            ways_to_prev2 = ways_to_prev1
            ways_to_prev1 = current_ways

        # ways_to_prev1 now stores ways(n)
        return ways_to_prev1
```

4. Complexity Analysis

Time Complexity

$O(n)$

We iterate from step 3 to step n , doing constant work at each step.

Space Complexity

$O(1)$

We only store a few variables regardless of the input size.

5. Alternative Approaches

1. Recursion

Directly use:

```
ways(n) = ways(n - 1) + ways(n - 2)
```

But plain recursion recomputes the same values many times.

Time complexity:

$O(2^n)$

This is inefficient.

2. Recursion with Memoization

Store already-computed results in a dictionary or list.

```
class Solution:
    def climbStairs(self, n: int) -> int:
        memo = {}

        def dp(step):
            if step == 1:
                return 1
            if step == 2:
                return 2

            if step in memo:
                return memo[step]

            memo[step] = dp(step - 1) + dp(step - 2)
            return memo[step]

        return dp(n)
```

Time complexity:

$O(n)$

Space complexity:

$O(n)$

3. Bottom-Up DP Array

Store all values in an array.

```
class Solution:
    def climbStairs(self, n: int) -> int:
        if n == 1:
            return 1

        dp = [0] * (n + 1)

        dp[1] = 1
        dp[2] = 2
```

```
for step in range(3, n + 1):  
    dp[step] = dp[step - 1] + dp[step - 2]  
  
return dp[n]
```

Time complexity:

$O(n)$

Space complexity:

$O(n)$

This is easier to understand but uses more memory than necessary.

Key Pattern

This problem is a classic example of:

Dynamic Programming + Fibonacci recurrence

Whenever a problem asks for the number of ways to reach a target using fixed step sizes, think about defining:

$dp[i]$ = number of ways to reach state i

Then build the recurrence from the possible previous states.